

CS3301 - Data Structures

Topic: topological sort

1. SUMMARY

Topological sort is a linear ordering of vertices in a directed acyclic graph (DAG) such that for every directed edge $u \rightarrow v$, vertex u comes before v in the ordering. The concept of topological sort is crucial in scheduling tasks that have dependencies on each other. Key principles from the reference include the definition of topological sort, its applications, and the importance of understanding graph terminology. The reference does not explicitly mention types and classifications of topological sort, but it highlights the significance of graph traversal methods, including DFS and BFS. Topological sort has numerous applications in real-world scenarios, such as job scheduling, course prerequisites, and network analysis. Important points highlighted in the reference include the need to understand basic data structures, graph representation, and traversal methods. The reference emphasizes the importance of understanding the theoretical implications of topological sort and its relation to other data structures. Topological sort is a fundamental concept in computer science, and its applications are diverse and widespread. The reference provides a comprehensive overview of the concept, including its definition, principles, and applications.

2. SHORT NOTES

Topological sort is a fundamental concept in graph theory, and it has numerous applications in computer science. A detailed definition of topological sort can be given as follows: given a directed acyclic graph (DAG) $G = (V, E)$, a topological sort is a linear ordering of the vertices in V such that for every directed edge (u, v) in E , vertex u comes before vertex v in the ordering. For example, consider a DAG with vertices A, B, C, D , and E , and edges $(A, B), (A, C), (B, D), (C, D)$, and (D, E) . A topological sort of this graph could be A, C, B, D, E . There are several types of topological sort, including Kahn's algorithm and DFS-based algorithms. The step-by-step working principle of topological sort involves selecting a vertex with no incoming edges, removing it from the graph, and repeating the process until all vertices have been removed. Important formulas and equations related to topological sort include the concept of in-degree and out-degree of a vertex. Diagrams can be used to illustrate the concept of topological sort, such as a graph with vertices and

edges. The advantages of topological sort include its ability to handle large graphs efficiently and its applications in real-world scenarios. However, topological sort has some disadvantages, such as its inability to handle graphs with cycles. Real-world applications of topological sort include job scheduling, course prerequisites, and network analysis. Topological sort can be compared to other related concepts, such as DFS and BFS, which are also used for graph traversal. The time and space complexity of topological sort depend on the algorithm used, but it is typically $O(V + E)$, where V is the number of vertices and E is the number of edges. Important terminology from the reference includes graph, vertex, edge, directed acyclic graph (DAG), and topological sort.

3. PRACTICAL / CODING EXAMPLES

C Implementation

```c

```
void topological_sort(graph *g) {
 int *in_degree = (int *) malloc(g->num_vertices * sizeof(int));
 for (int i = 0; i < g->num_vertices; i++) {
 in_degree[i] = 0;
 }
 for (int i = 0; i < g->num_vertices; i++) {
 for (int j = 0; j < g->num_vertices; j++) {
 if (g->adj_matrix[i][j] == 1) {
 in_degree[j]++;
 }
 }
 }
 queue *q = create_queue();
 for (int i = 0; i < g->num_vertices; i++) {
 if (in_degree[i] == 0) {
 enqueue(q, i);
 }
 }
}
```

```

}
while (!is_empty(q)) {
 int vertex = dequeue(q);
 printf("%d ", vertex);
 for (int i = 0; i < g->num_vertices; i++) {
 if (g->adj_matrix[vertex][i] == 1) {
 in_degree[i]--;
 if (in_degree[i] == 0) {
 enqueue(q, i);
 }
 }
 }
}
}
}
...

```

### Python Implementation

```
```python
```

```
from collections import defaultdict, deque
```

```
def topological_sort(graph):
```

```
    in_degree = {vertex: 0 for vertex in graph}
```

```
    for vertex in graph:
```

```
        for neighbor in graph[vertex]:
```

```
            in_degree[neighbor] += 1
```

```
    queue = deque([vertex for vertex in in_degree if in_degree[vertex] == 0])
```

```
    sorted_vertices = []
```

```
    while queue:
```

```
        vertex = queue.popleft()
```

```
        sorted_vertices.append(vertex)
```

```
        for neighbor in graph[vertex]:
```

```
in_degree[neighbor] -= 1
if in_degree[neighbor] == 0:
    queue.append(neighbor)
return sorted_vertices
```

Algorithm

1. Create a graph and calculate the in-degree of each vertex

2. Create a queue and enqueue all vertices with in-degree 0

3. While the queue is not empty, dequeue a vertex and print it

4. For each neighbor of the dequeued vertex, decrease its in-degree by 1

5. If the in-degree of a neighbor becomes 0, enqueue it

Time and Space Complexity

Time complexity: $O(V + E)$, where V is the number of vertices and E is the number of edges

Space complexity: $O(V + E)$, where V is the number of vertices and E is the number of edges

...

4. IMPORTANT QUESTIONS

****Part-A****

Q1. Define topological sort and list its key characteristics.

Answer: Topological sort is a linear ordering of vertices in a directed acyclic graph (DAG) such that for every directed edge $u \rightarrow v$, vertex u comes before v in the ordering. The key characteristics of topological sort are: (1) it is a linear ordering, (2) it is defined for DAGs only, (3) it satisfies the condition that for every edge $u \rightarrow v$, u comes before v in the ordering, and (4) it is not unique, as there can be multiple valid topological orderings for a given DAG.

Q2. Compare topological sort with breadth-first search (BFS).

Answer: Topological sort and BFS are both graph traversal algorithms, but they serve different purposes. Topological sort is used to order the vertices of a DAG such that for every edge $u \rightarrow v$, u comes before v ,

whereas BFS is used to traverse a graph level by level, starting from a given source vertex. While BFS can be used to detect cycles in a graph, topological sort is used to order the vertices of a DAG.

Q3. Why is topological sort important in computer science?

Answer: Topological sort is important in computer science because it has numerous applications in various fields, such as: (1) scheduling tasks with dependencies, (2) course prerequisites in academic programs, (3) data processing pipelines, and (4) compiler design. Topological sort helps to ensure that tasks are executed in a valid order, preventing conflicts and deadlocks.

Q4. Write a short note on the applications of topological sort.

Answer: Topological sort has numerous applications, including: (1) scheduling tasks with dependencies, (2) course prerequisites in academic programs, (3) data processing pipelines, (4) compiler design, (5) network topology analysis, and (6) resource allocation. In each of these applications, topological sort helps to ensure that tasks are executed in a valid order, preventing conflicts and deadlocks.

Q5. Give an example of topological sort in real-world applications.

Answer: A classic example of topological sort is in course scheduling. Suppose we have a set of courses with prerequisites, and we want to schedule the courses such that a student can take a course only after completing all its prerequisites. A topological sort can be used to order the courses such that for every course u with a prerequisite v , course v comes before course u in the ordering.

****Part-B****

Q1. Explain topological sort in detail with neat diagram and suitable examples.

Answer:

Topological sort is a linear ordering of vertices in a directed acyclic graph (DAG) such that for every directed edge $u \rightarrow v$, vertex u comes before v in the ordering. There are two types of topological sort: (1) Kahn's algorithm, which uses a queue to keep track of vertices with no incoming edges, and (2) DFS-based algorithm, which uses a stack to keep track of vertices with no incoming edges.

Here is a step-by-step explanation of Kahn's algorithm:

1. Initialize an empty queue and an empty ordering.
2. Find all vertices with no incoming edges and add them to the queue.
3. While the queue is not empty, dequeue a vertex and add it to the ordering.
4. For each neighbor of the dequeued vertex, decrement its incoming edge count. If the count becomes zero, add the neighbor to the queue.
5. Repeat steps 3-4 until the queue is empty.

The diagram shows a sample DAG with vertices A, B, C, D, and E. The topological sort ordering is A, B, C, D, E.

Topological sort has numerous applications, including: (1) scheduling tasks with dependencies, (2) course prerequisites in academic programs, (3) data processing pipelines, and (4) compiler design. The advantages of topological sort are: (1) it helps to ensure that tasks are executed in a valid order, preventing conflicts and deadlocks, and (2) it can be used to detect cycles in a graph. The disadvantages of topological sort are: (1) it can be computationally expensive for large graphs, and (2) it may not always produce a unique ordering.

In conclusion, topological sort is an important concept in computer science with numerous applications. It helps to ensure that tasks are executed in a valid order, preventing conflicts and deadlocks.

Q2. Describe the types of topological sort with working principle and examples.

Answer:

There are two types of topological sort: (1) Kahn's algorithm, which uses a queue to keep track of vertices with no incoming edges, and (2) DFS-based algorithm, which uses a stack to keep track of vertices with no incoming edges.

Kahn's algorithm works as follows:

1. Initialize an empty queue and an empty ordering.
2. Find all vertices with no incoming edges and add them to the queue.
3. While the queue is not empty, dequeue a vertex and add it to the ordering.
4. For each neighbor of the dequeued vertex, decrement its incoming edge count. If the count becomes zero, add the neighbor to the queue.

5. Repeat steps 3-4 until the queue is empty.

The DFS-based algorithm works as follows:

1. Initialize an empty stack and an empty ordering.
2. Find all vertices with no incoming edges and push them onto the stack.
3. While the stack is not empty, pop a vertex and add it to the ordering.
4. For each neighbor of the popped vertex, decrement its incoming edge count. If the count becomes zero, push the neighbor onto the stack.
5. Repeat steps 3-4 until the stack is empty.

Here is a comparison table of the two types of topological sort:

Algorithm	Time Complexity	Space Complexity
---	---	---
Kahn's algorithm	$O(V + E)$	$O(V)$
DFS-based algorithm	$O(V + E)$	$O(V)$

The real-world applications of topological sort include: (1) scheduling tasks with dependencies, (2) course prerequisites in academic programs, (3) data processing pipelines, and (4) compiler design.

Q3. Compare topological sort with related concepts. Discuss advantages and applications.

Answer:

Topological sort is related to other graph traversal algorithms, such as breadth-first search (BFS) and depth-first search (DFS). While BFS is used to traverse a graph level by level, starting from a given source vertex, DFS is used to traverse a graph by exploring as far as possible along each branch before backtracking.

Here is a comparison table of topological sort with BFS and DFS:

Algorithm	Purpose	Time Complexity	Space Complexity
---	---	---	---
Topological sort	Ordering vertices in a DAG	$O(V + E)$	$O(V)$
BFS	Traversing a graph level by level	$O(V + E)$	$O(V)$

| DFS | Traversing a graph by exploring as far as possible along each branch | $O(V + E)$ | $O(V)$ |

Topological sort has advantages over BFS and DFS in that it can be used to order the vertices of a DAG, which is not possible with BFS and DFS. However, topological sort can be computationally expensive for large graphs, and it may not always produce a unique ordering.

The applications of topological sort include: (1) scheduling tasks with dependencies, (2) course prerequisites in academic programs, (3) data processing pipelines, and (4) compiler design. In each of these applications, topological sort helps to ensure that tasks are executed in a valid order, preventing conflicts and deadlocks.

****Part-C****

Q1. Elaborate on topological sort covering theory, implementation, real-world applications, and future scope.

Answer:

****Section 1: Deep theoretical background****

Topological sort is a linear ordering of vertices in a directed acyclic graph (DAG) such that for every directed edge $u \rightarrow v$, vertex u comes before v in the ordering. The theory behind topological sort is based on the concept of a DAG, which is a graph with no cycles. A topological sort is possible only if the graph is a DAG.

****Section 2: How it is implemented/used in practice****

Topological sort is implemented using two main algorithms: Kahn's algorithm and DFS-based algorithm. Kahn's algorithm uses a queue to keep track of vertices with no incoming edges, while DFS-based algorithm uses a stack to keep track of vertices with no incoming edges. In practice, topological sort is used in various applications, such as scheduling tasks with dependencies, course prerequisites in academic programs, data processing pipelines, and compiler design.

****Section 3: Real-world case studies or applications****

Here are three real-world case studies of topological sort:

1. Scheduling tasks with dependencies: Topological sort is used to schedule tasks with dependencies in a manufacturing process. The tasks are represented as vertices in a graph, and the dependencies between tasks are represented as edges. A topological sort is used to order the tasks such that for every dependency

$u \rightarrow v$, task u is executed before task v .

2. Course prerequisites in academic programs: Topological sort is used to order the courses in an academic program such that for every course u with a prerequisite v , course v is taken before course u .

3. Data processing pipelines: Topological sort is used to order the data processing tasks in a pipeline such that for every task u with a dependency v , task v is executed before task u .

****Section 4: Limitations and challenges****

The limitations of topological sort are: (1) it can be computationally expensive for large graphs, and (2) it may not always produce a unique ordering. The challenges of topological sort are: (1) handling cycles in the graph, and (2) handling vertices with no incoming edges.

****Section 5: Future scope and improvements****

The future scope of topological sort includes: (1) developing more efficient algorithms for large graphs, and (2) developing algorithms for handling cycles in the graph. The improvements of topological sort include: (1) using parallel processing to speed up the computation, and (2) using machine learning algorithms to predict the dependencies between tasks.

In conclusion, topological sort is an important concept in computer science with numerous applications. It helps to ensure that tasks are executed in a valid order, preventing conflicts and deadlocks.

Q2. Critically analyze topological sort. Compare approaches, evaluate trade-offs, and justify with examples.

Answer:

****Section 1: Overview of the concept and its importance****

Topological sort is a linear ordering of vertices in a directed acyclic graph (DAG) such that for every directed edge $u \rightarrow v$, vertex u comes before v in the ordering. Topological sort is important in computer science because it has numerous applications in various fields, such as scheduling tasks with dependencies, course prerequisites in academic programs, data processing pipelines, and compiler design.

****Section 2: Different approaches/techniques used****

There are two main approaches to topological sort: (1) Kahn's algorithm, which uses a queue to keep track of vertices with no incoming edges, and (2) DFS-based algorithm, which uses a stack to keep track of vertices

with no incoming edges.

****Section 3: Comparison of approaches with a table****

Here is a comparison table of the two approaches:

Algorithm	Time Complexity	Space Complexity
---	---	---
Kahn's algorithm	$O(V + E)$	$O(V)$
DFS-based algorithm	$O(V + E)$	$O(V)$

****Section 4: Trade-off analysis (time, space, cost, complexity)****

The trade-offs of topological sort are: (1) time complexity vs. space complexity, and (2) computational cost vs. accuracy. The time complexity of topological sort is $O(V + E)$, which is the same as the space complexity. However, the computational cost of topological sort can be high for large graphs

5. MCQs

Q1. What is the primary purpose of topological sort?

- a) To find the shortest path between two vertices
- b) To find the minimum spanning tree of a graph
- c) To order the vertices of a directed acyclic graph (DAG) such that for every directed edge $u \rightarrow v$, vertex u comes before v in the ordering
- d) To find the maximum flow in a flow network

Answer: c) To order the vertices of a directed acyclic graph (DAG) such that for every directed edge $u \rightarrow v$, vertex u comes before v in the ordering

Explanation: Topological sort is used to order the vertices of a DAG such that for every directed edge $u \rightarrow v$, vertex u comes before v in the ordering.

Q2. Which of the following is a type of topological sort?

- a) Kahn's algorithm
- b) DFS-based algorithm
- c) Both a and b
- d) None of the above

Answer: c) Both a and b

Explanation: Both Kahn's algorithm and DFS-based algorithm are types of topological sort.

Q3. What is the time complexity of topological sort?

- a) $O(V^2)$
- b) $O(V + E)$
- c) $O(E^2)$
- d) $O(V * E)$

Answer: b) $O(V + E)$

Explanation: The time complexity of topological sort is $O(V + E)$, where V is the number of vertices and E is the number of edges.

Q4. What is the space complexity of topological sort?

- a) $O(V^2)$
- b) $O(V + E)$
- c) $O(E^2)$
- d) $O(V * E)$

Answer: b) $O(V + E)$

Explanation: The space complexity of topological sort is $O(V + E)$, where V is the number of vertices and E is the number of edges.

Q5. What is an application of topological sort?

- a) Job scheduling
- b) Course prerequisites
- c) Network analysis
- d) All of the above

Answer: d) All of the above

Explanation: Topological sort has numerous applications, including job scheduling, course prerequisites, and network analysis.

7. YOUTUBE LINKS

- [Topological Sort Tutorial](https://www.youtube.com/results?search_query=topological+sort+tutorial)
- [Graph Theory for Beginners](https://www.youtube.com/results?search_query=graph+theory+for+beginners)
- [Topological Sort Algorithm](https://www.youtube.com/results?search_query=topological+sort+algorithm)
- [Directed Acyclic Graphs (DAGs)](https://www.youtube.com/results?search_query=directed+acyclic+graphs)
- [Applications of Topological Sort](https://www.youtube.com/results?search_query=applications+of+topological+sort)